

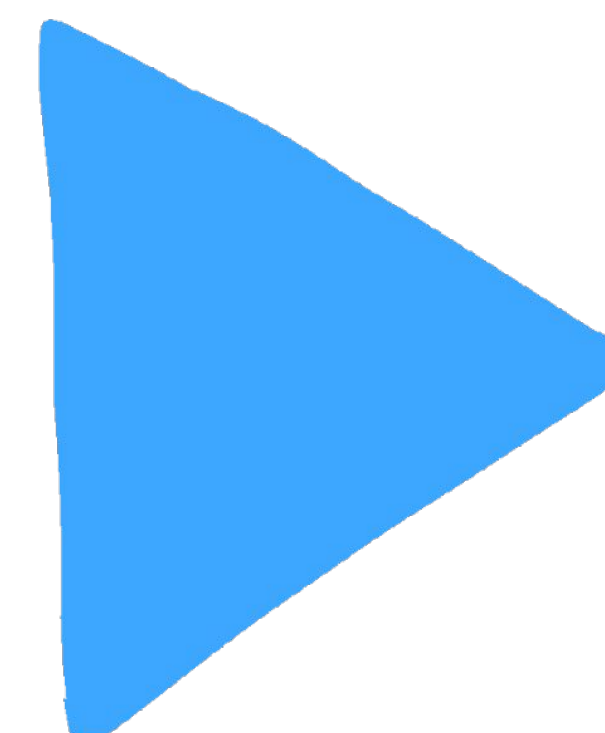
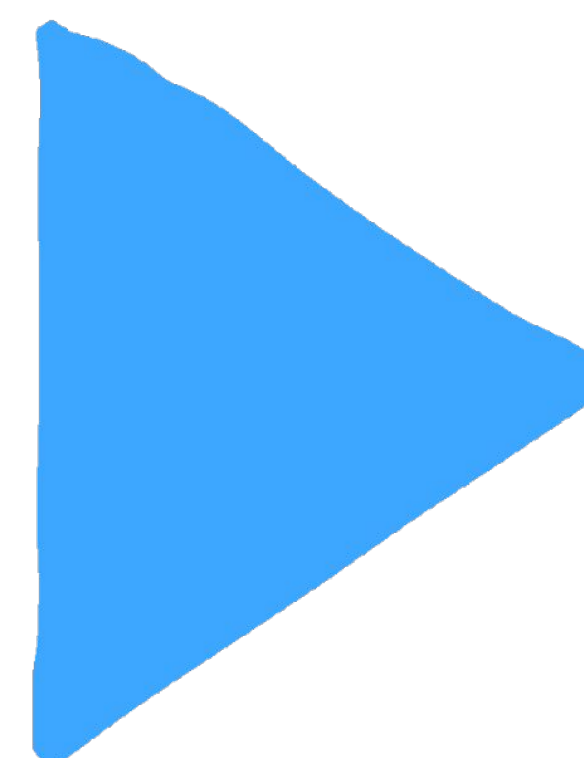


AI engineering tools 5





**Это полностью
прикладной курс.**





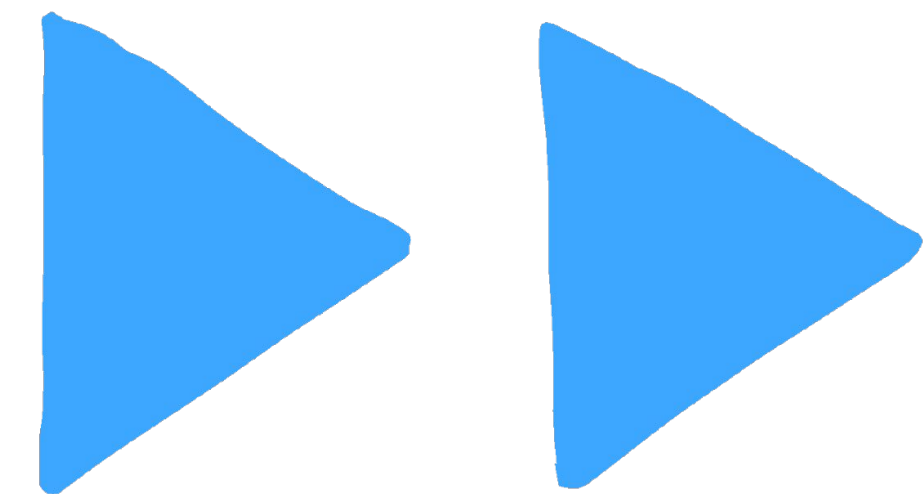
Шаповалов Александр

aTCL @ Internal automation

- Вижу, как медленно внедряется AI в бигтехе
- tg: @rostish

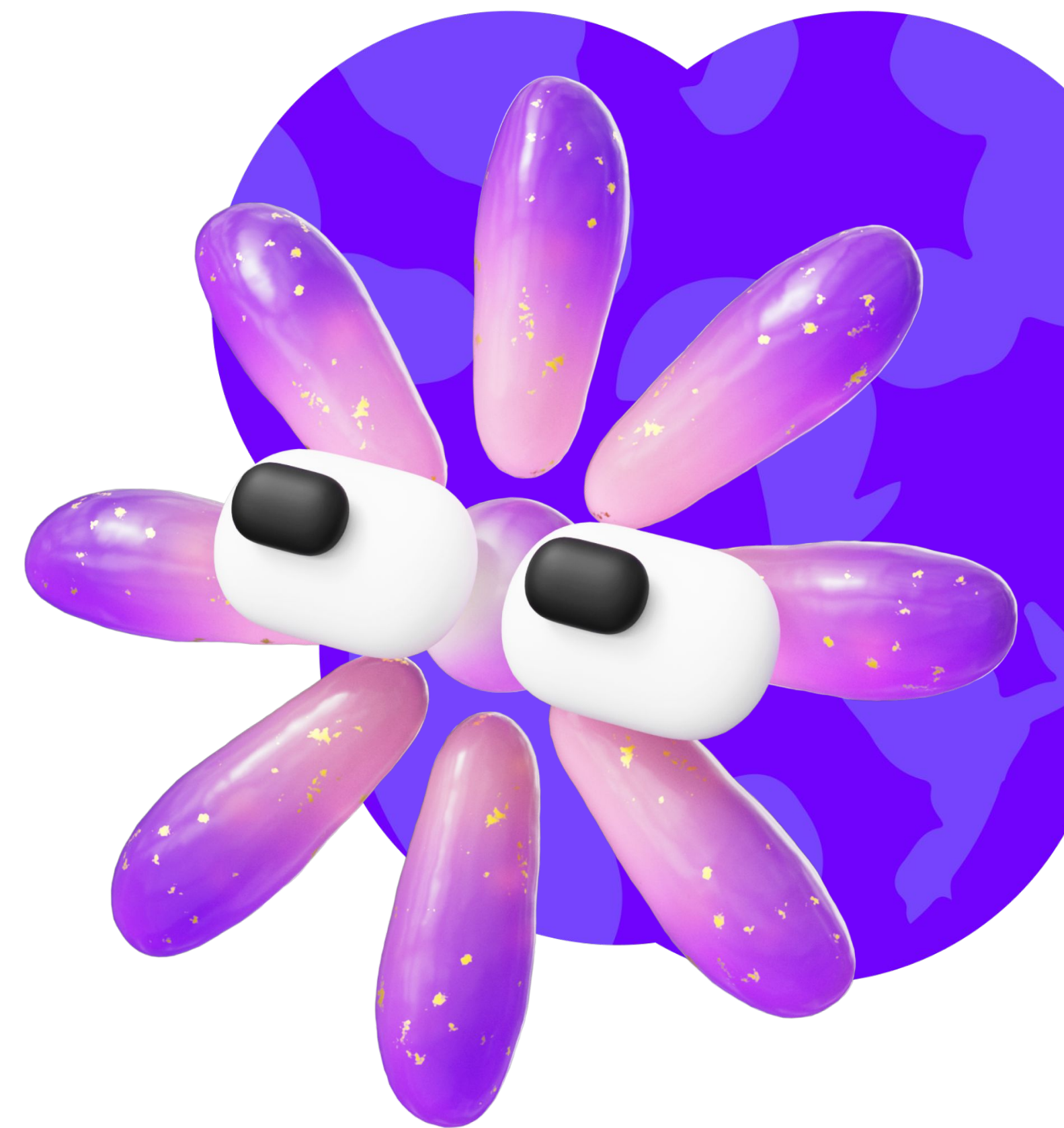
05

Спец-driven разработка с AI-агентами



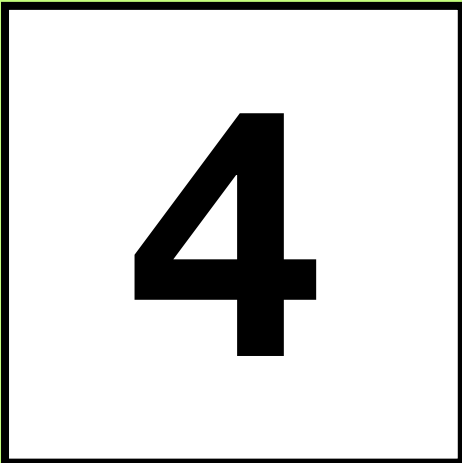
Сегодня узнаете

1. Немного про прикладную эффективность AI в цифрах
2. Spec driven



R1. Немного про эффективность

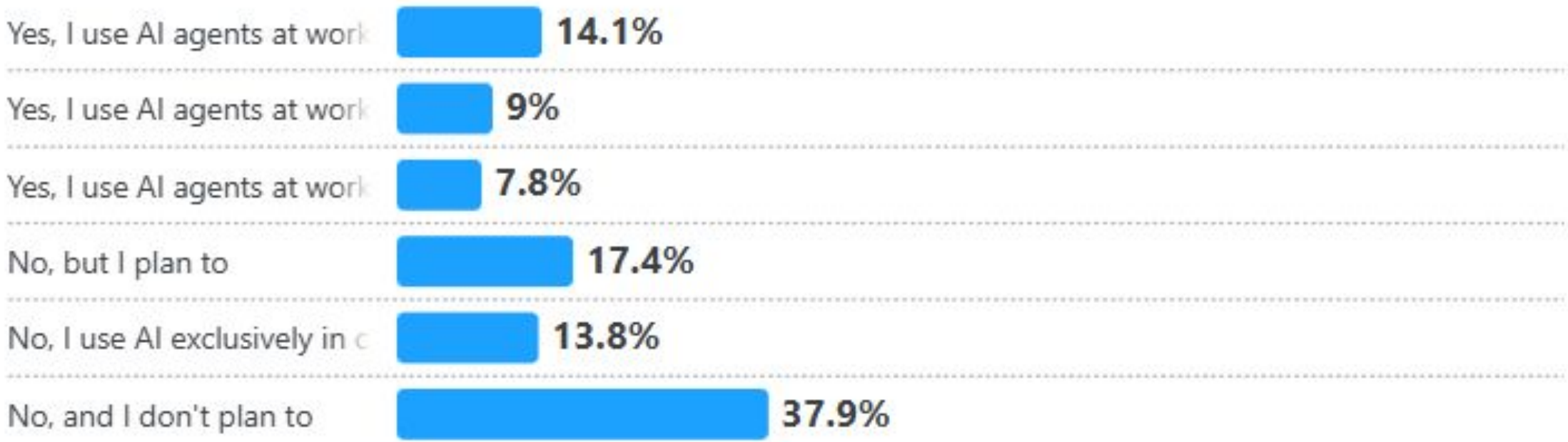
1. Green field vs Brown field



Greenfield/Brownfield

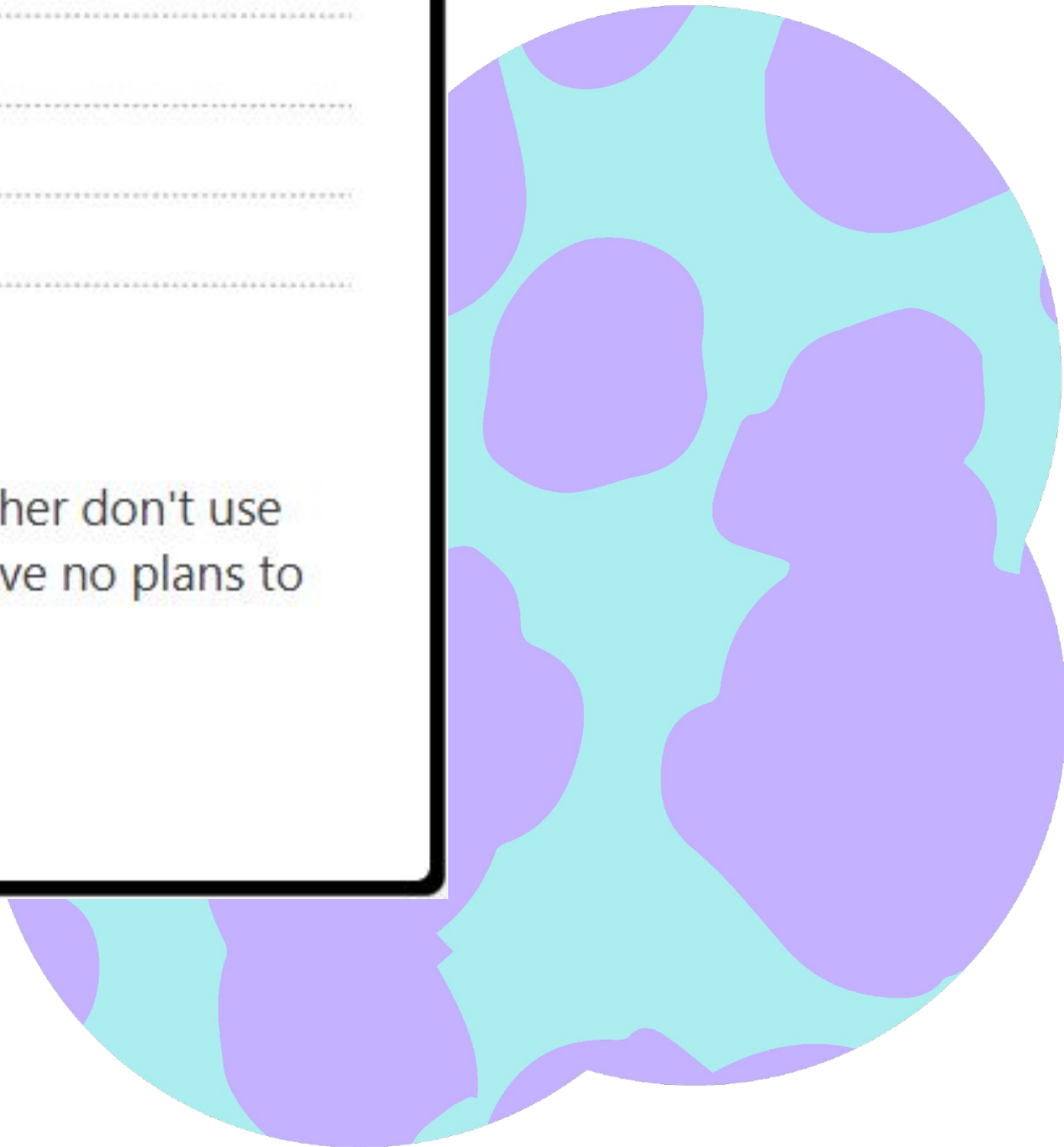
AI → AI Agents

A majority of developers don't use AI agents



AI agents are not yet mainstream. A majority of developers (52%) either don't use agents or stick to simpler AI tools, and a significant portion (38%) have no plans to adopt them.

AI agents →



Greenfield/Brownfield

Is AI a threat to your job?

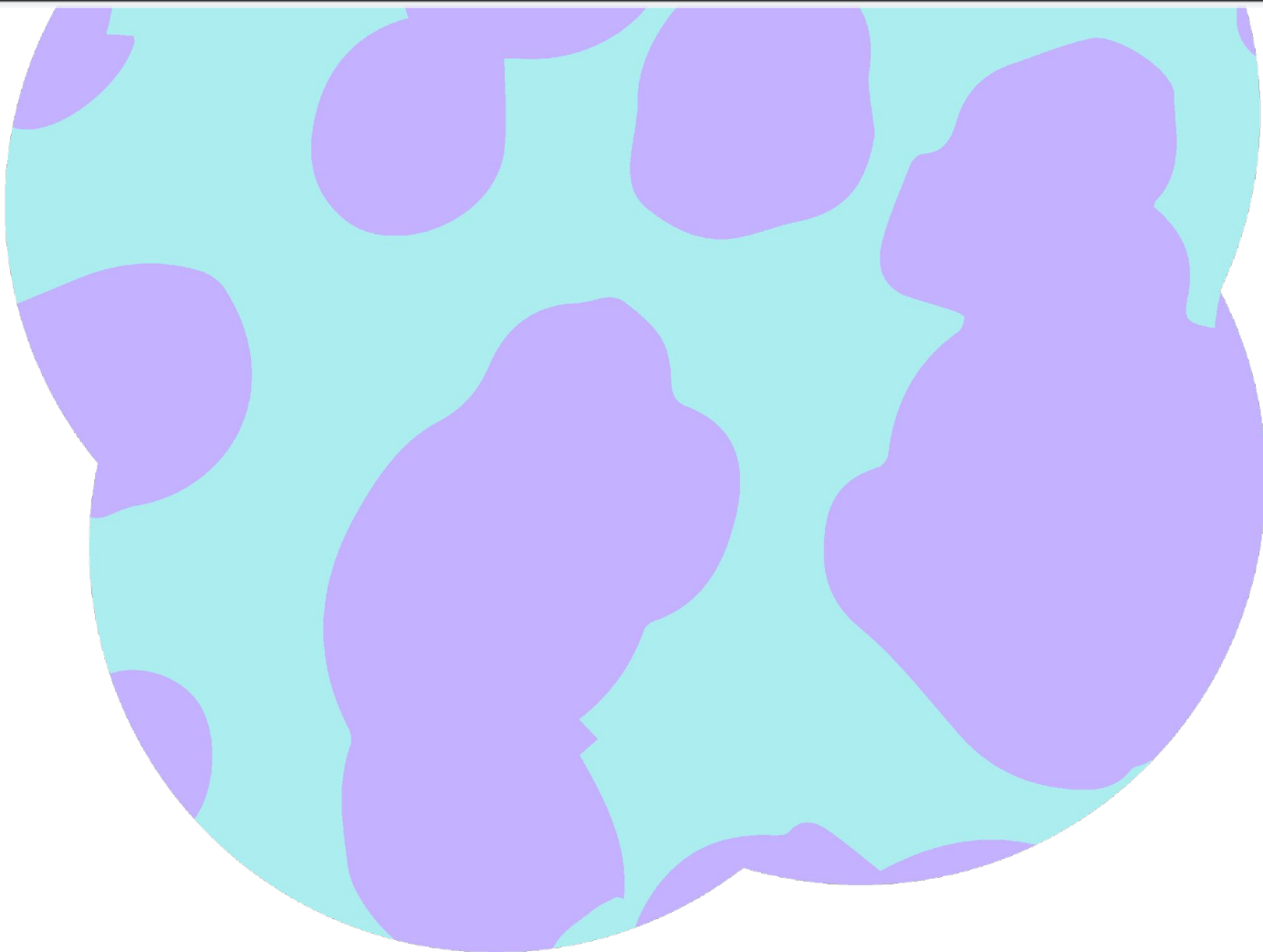
Confidence is slipping among developers when it comes to the perceived threat of AI to their livelihood. 64% believe AI is not a threat to their job, a decrease from 68% last year.

Do you believe AI is a threat to your current job?

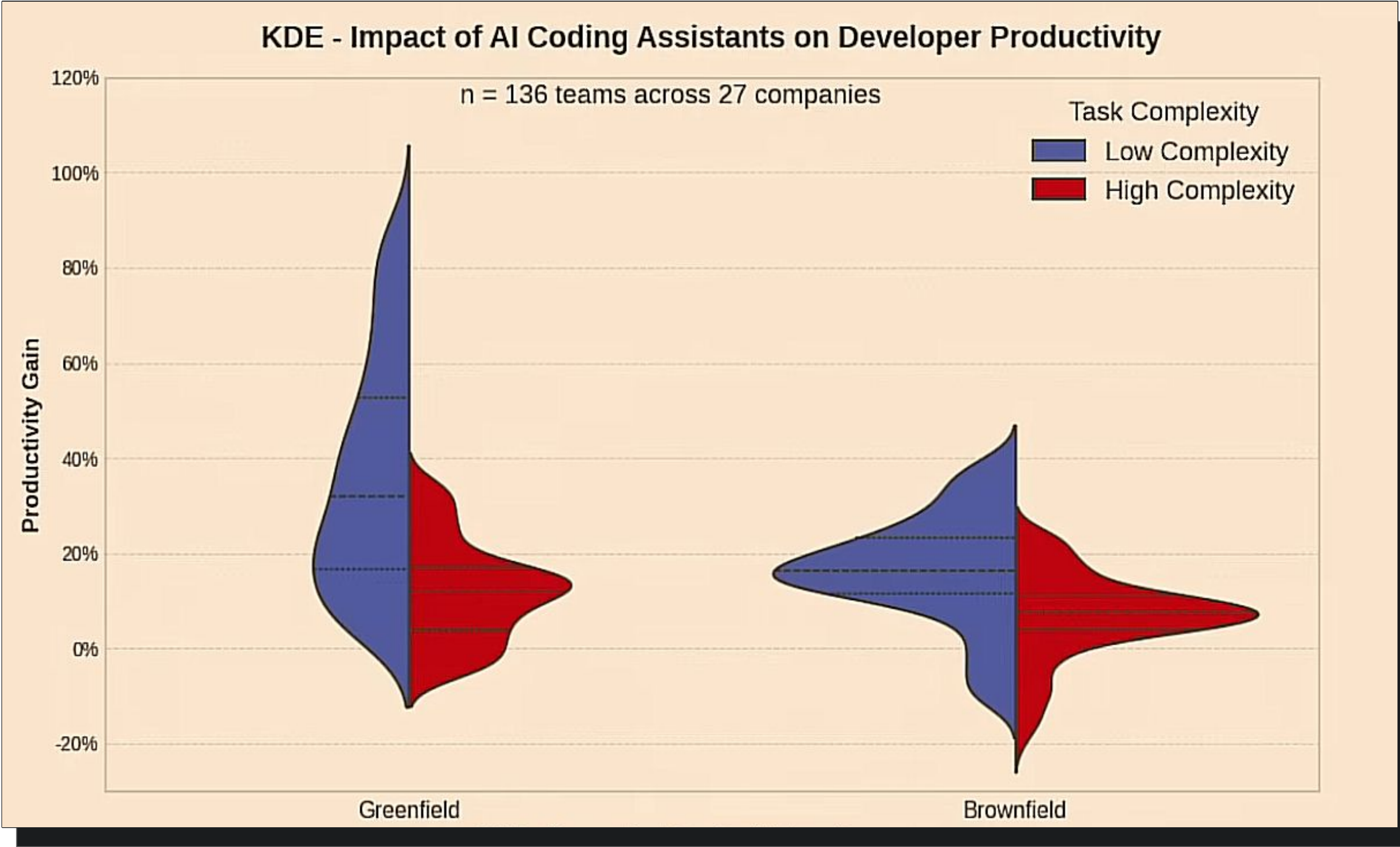


Download Share

Responses: 36,000(73.4%)



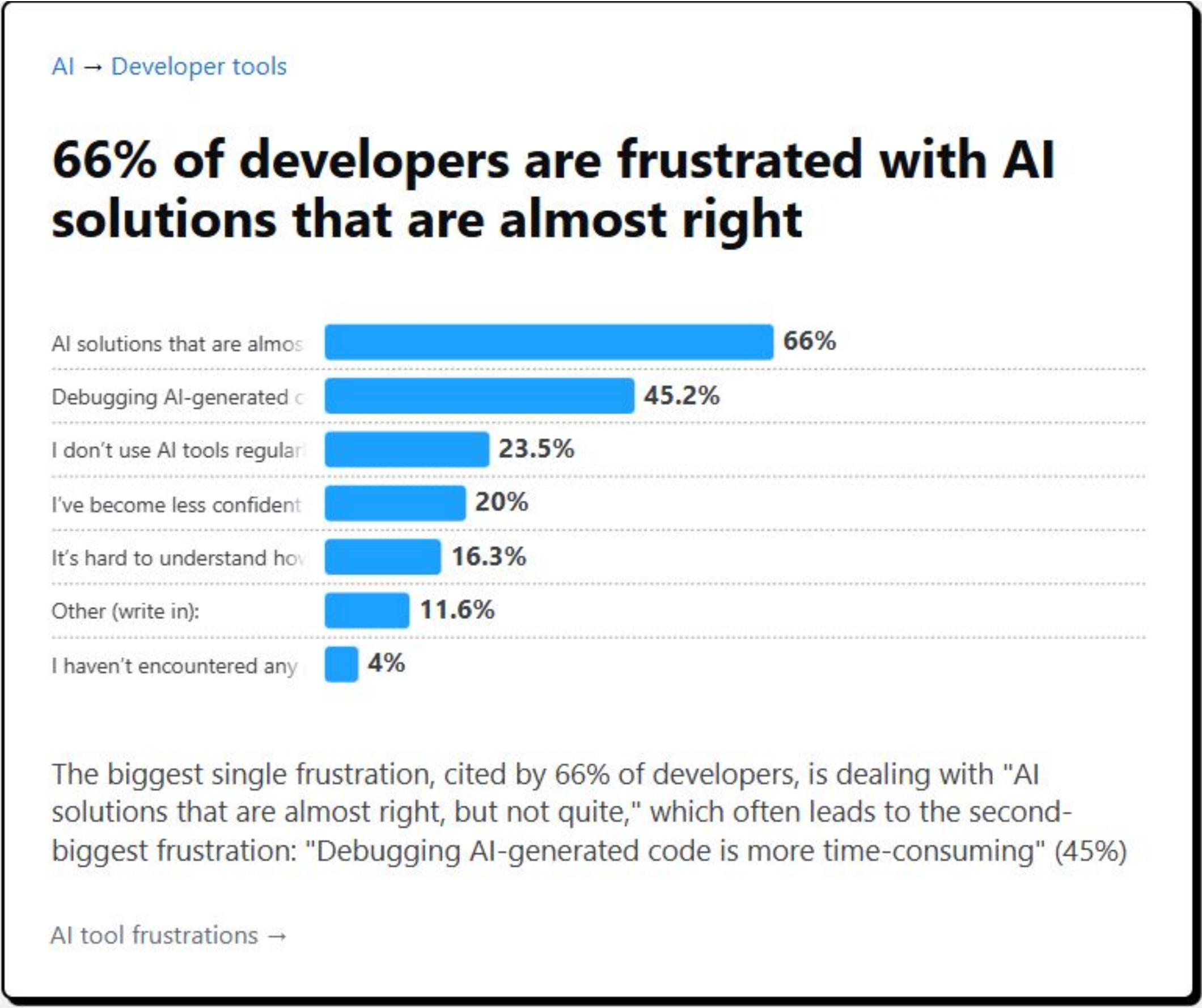
Greenfield/Brownfield



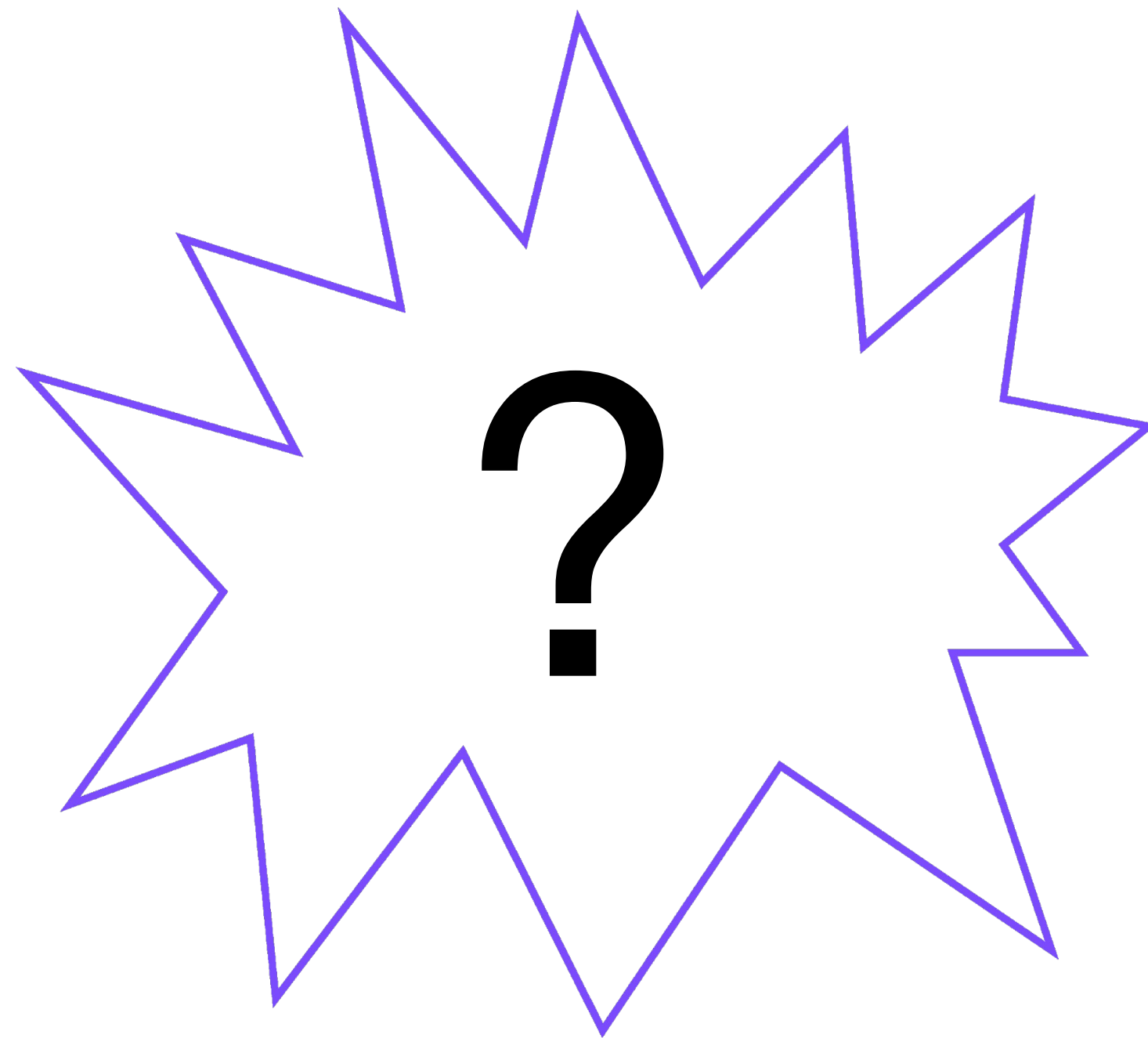
Greenfield/Brownfield



Greenfield/Brownfield



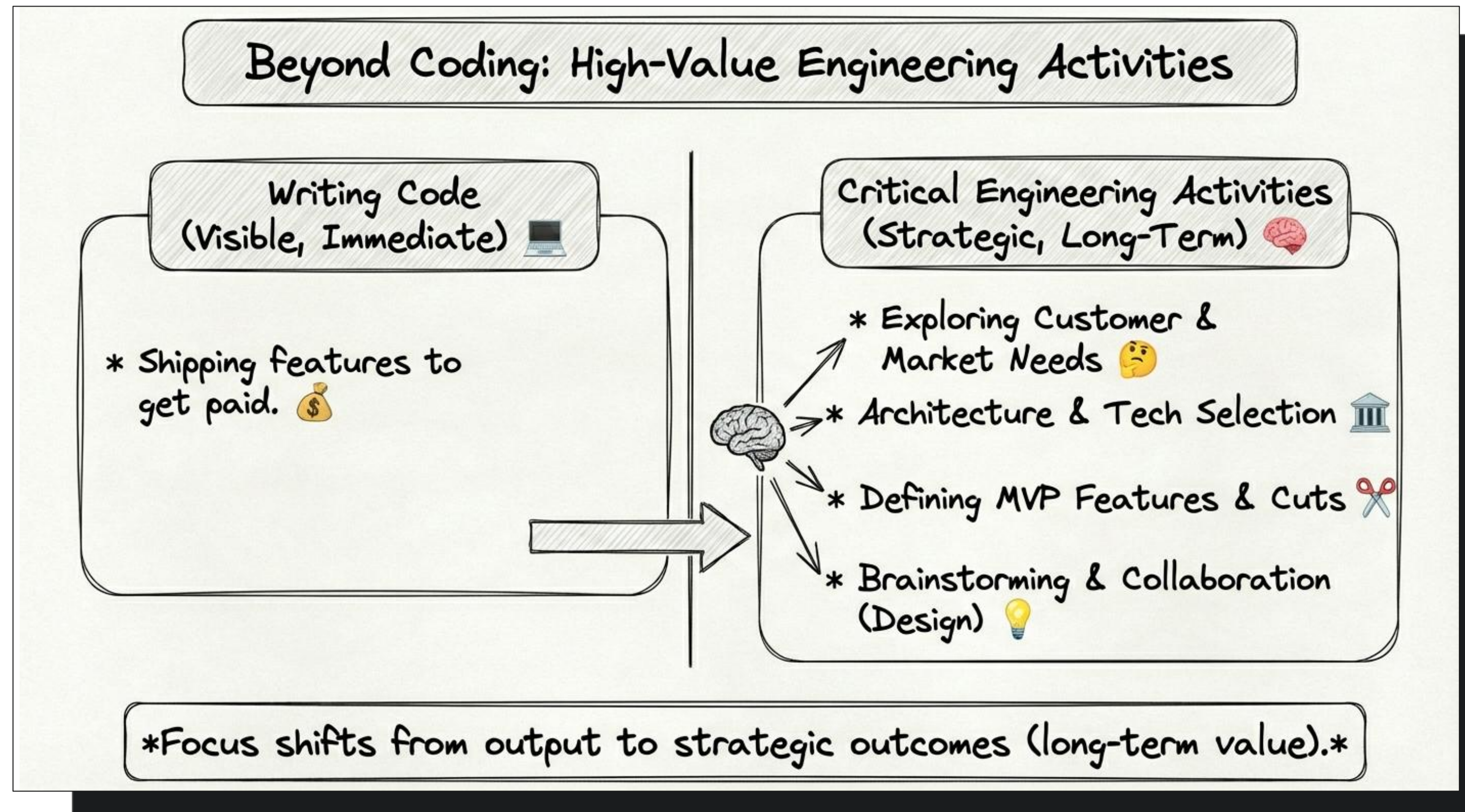
Почему нет кратного роста?



Почему нет кратного роста?

**An engineer's goal
is not writing code**

Почему нет кратного роста?



Почему нет кратного роста?

<https://juraj.blog/p/100x-software-engineering>



Почему нет кратного роста?

<https://survey.stackoverflow.co/2025/>



P2.Spec Driven Development

1. Generations of programming languages and SDD Layers
2. Варианты реализации

2

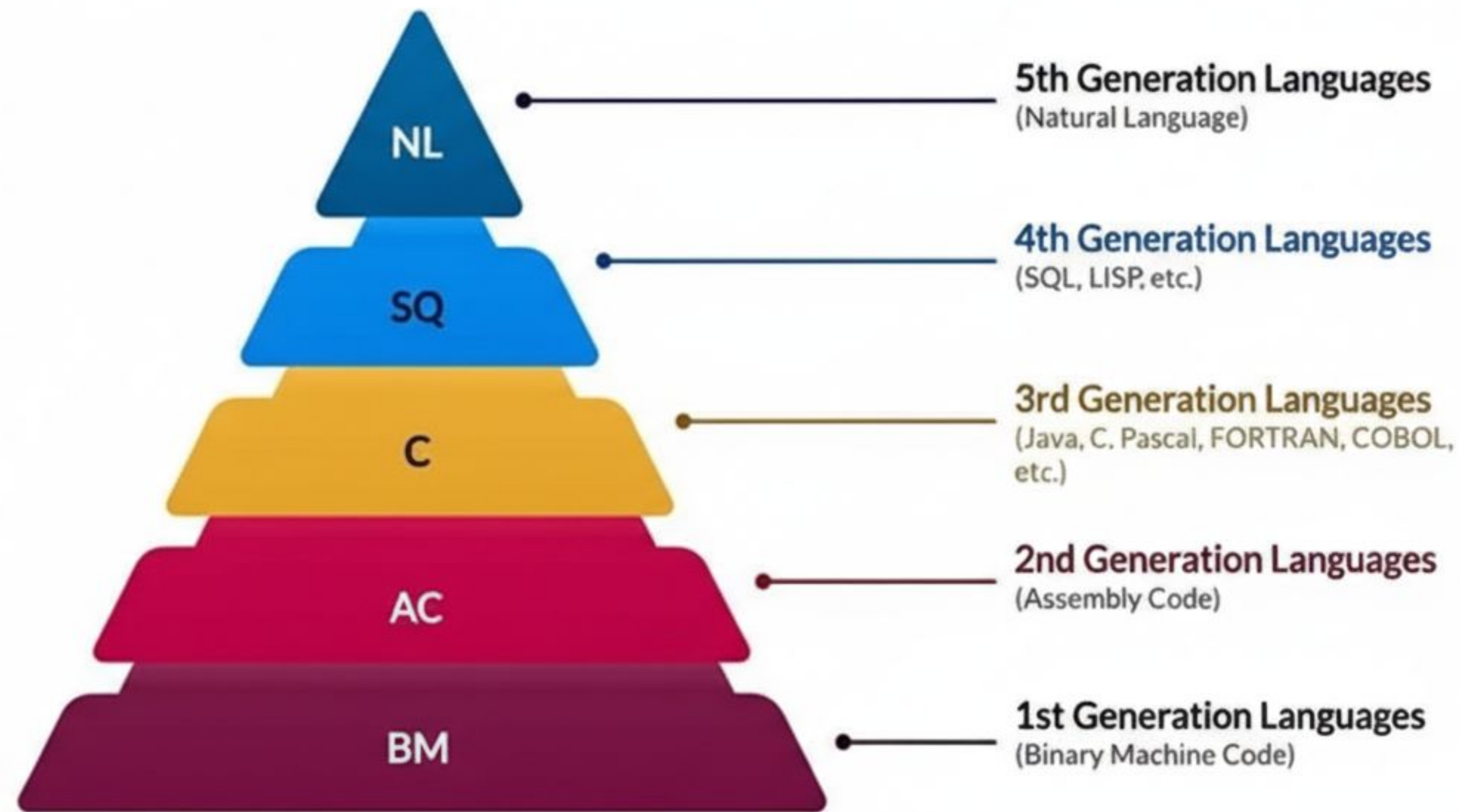


2

3

4

Generations of programming languages



Methodologies



Waterfall

Feature Driven Development

Rapid Application Development

Test Driven Development

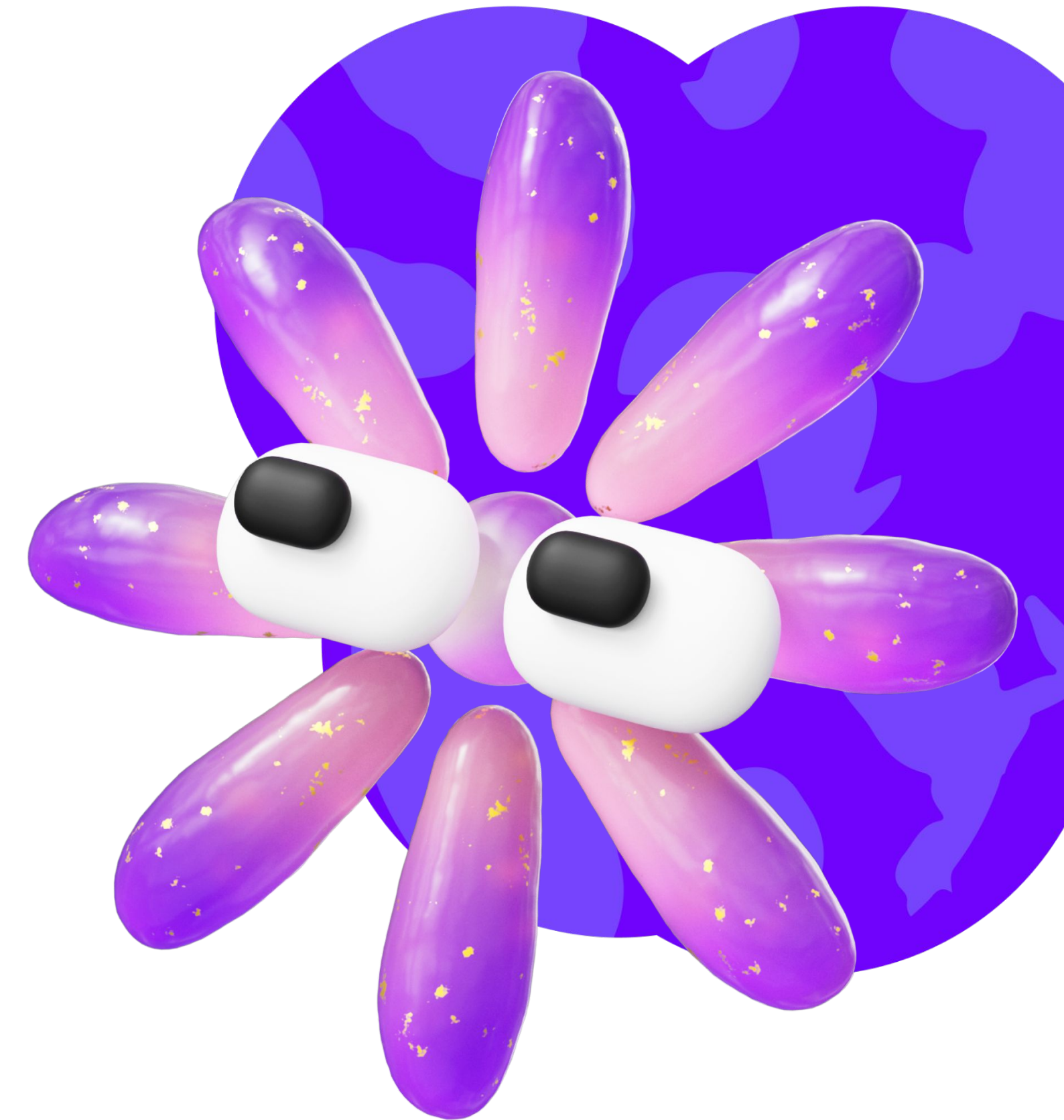
Extreme Programming

Iterative and Incremental Development

Generations of programming languages

Традиционные архитектуры полагаются на исходный код. SDD полагается на спецификации, позволяя нам определять такие артефакты, как:

- Интерфейсные контракты (возможности, входы/выходы, гарантии поведения)
- Схемы данных и инварианты (структура, ограничения, правила валидации)
- Топологии событий (разрешенные потоки, последовательность, семантика распространения)
- Границы безопасности (идентичность, зоны доверия, обеспечение соблюдения политик)
- Правила совместимости (как обратной, так и прямой)
- Семантика версионирования (эволюция, амортизация, миграция)
- Ограничения ресурсов и производительности (задержка, пропускная способность, затраты)



Generations of programming languages

Architecture is no longer advisory; it now becomes enforceable and executable.



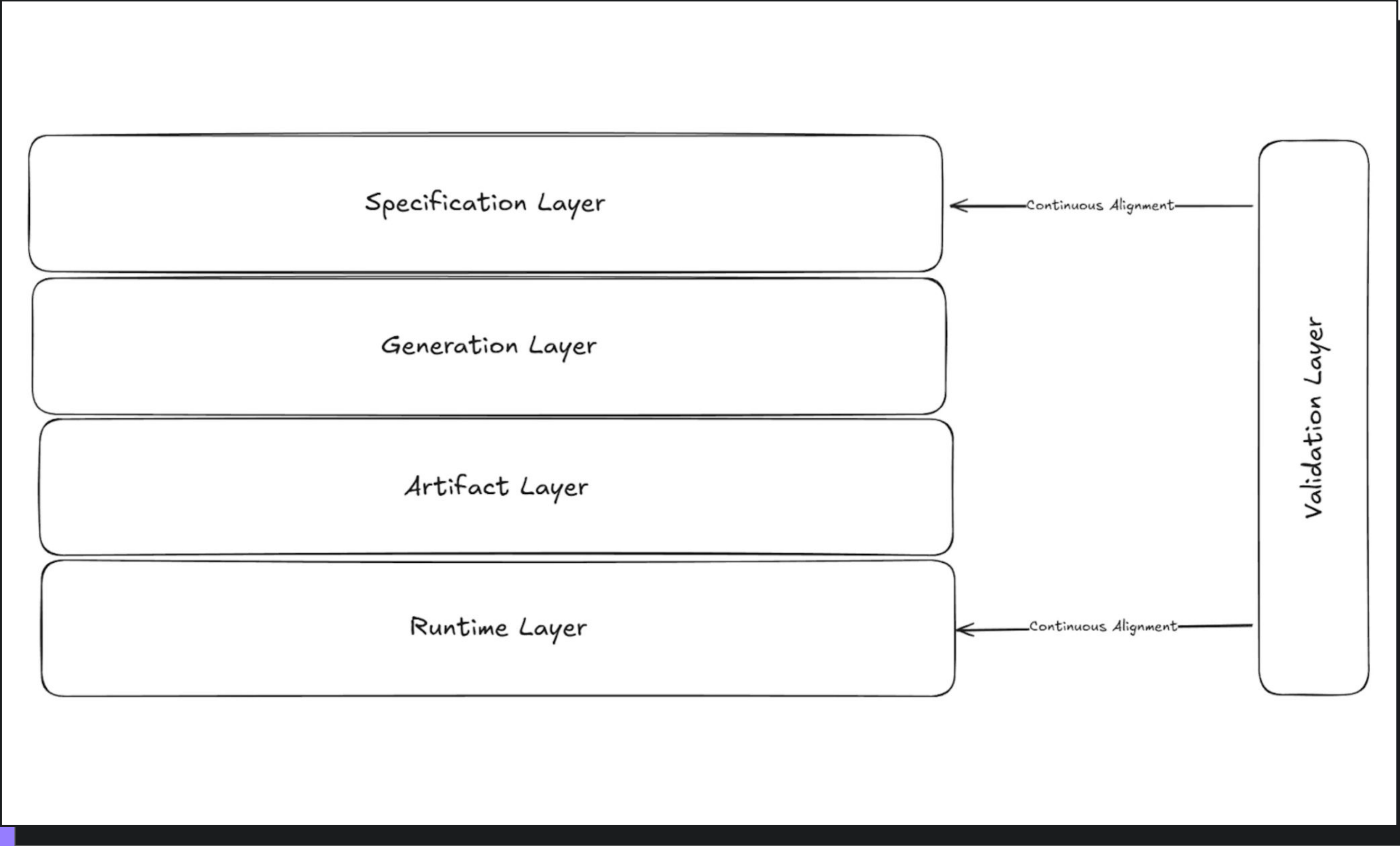
Methodologies

What is SDD?

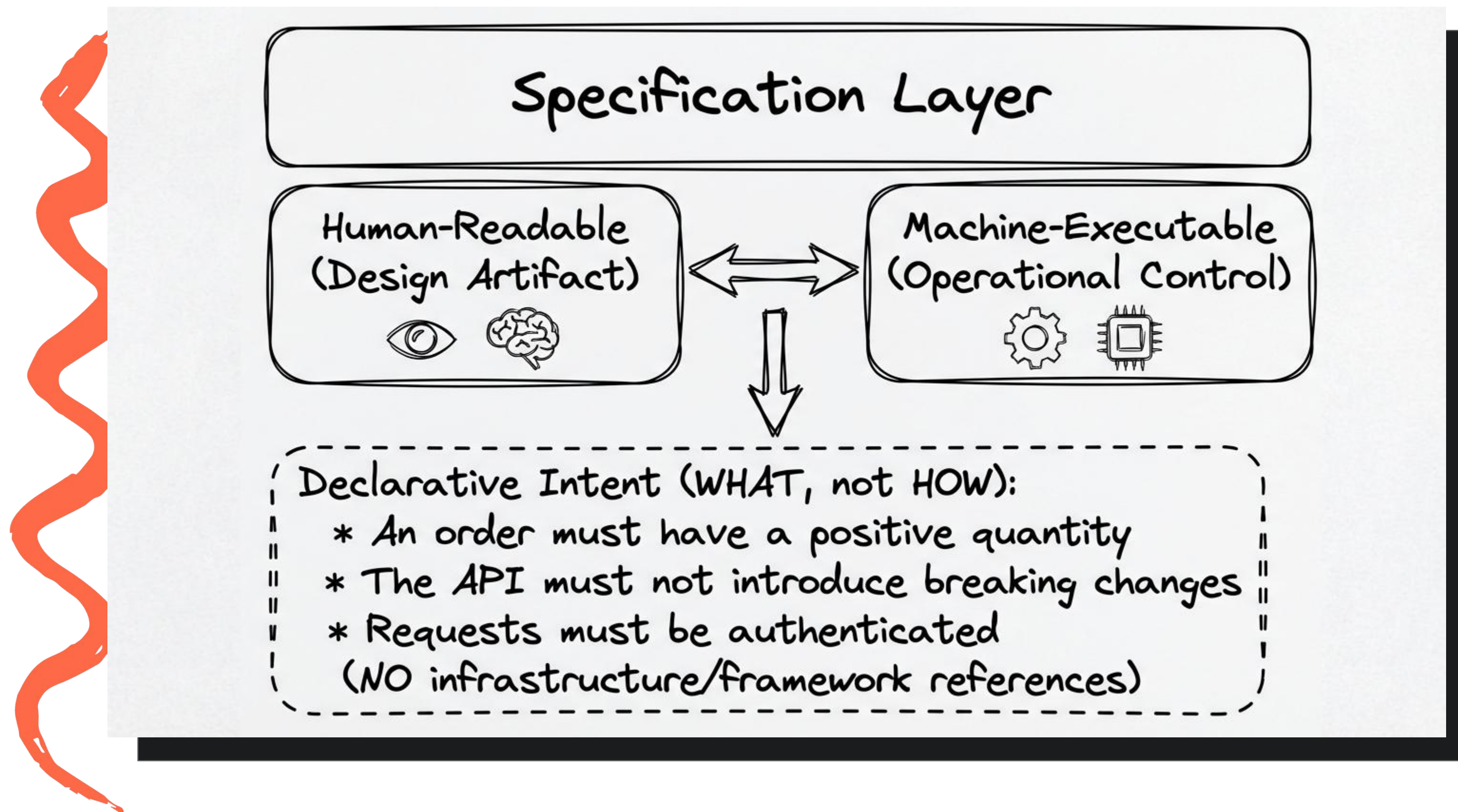
Spec Driven Development (SDD) is a software development methodology in which the specification (the formal description of what the system should do) is the primary source of truth that drives the entire development process.

SDD flips the script: specifications, not code, dictate behavior.

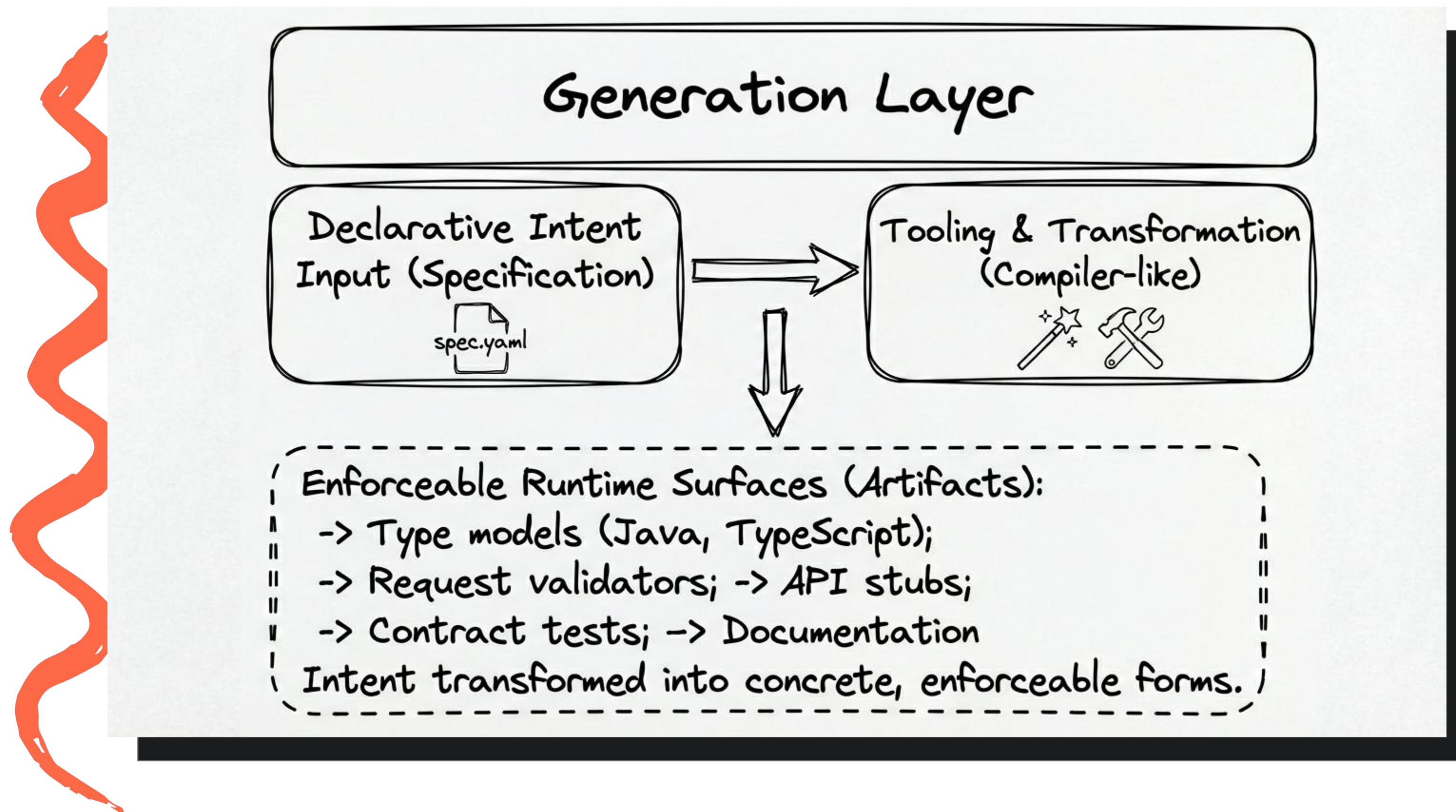
SDD Layers



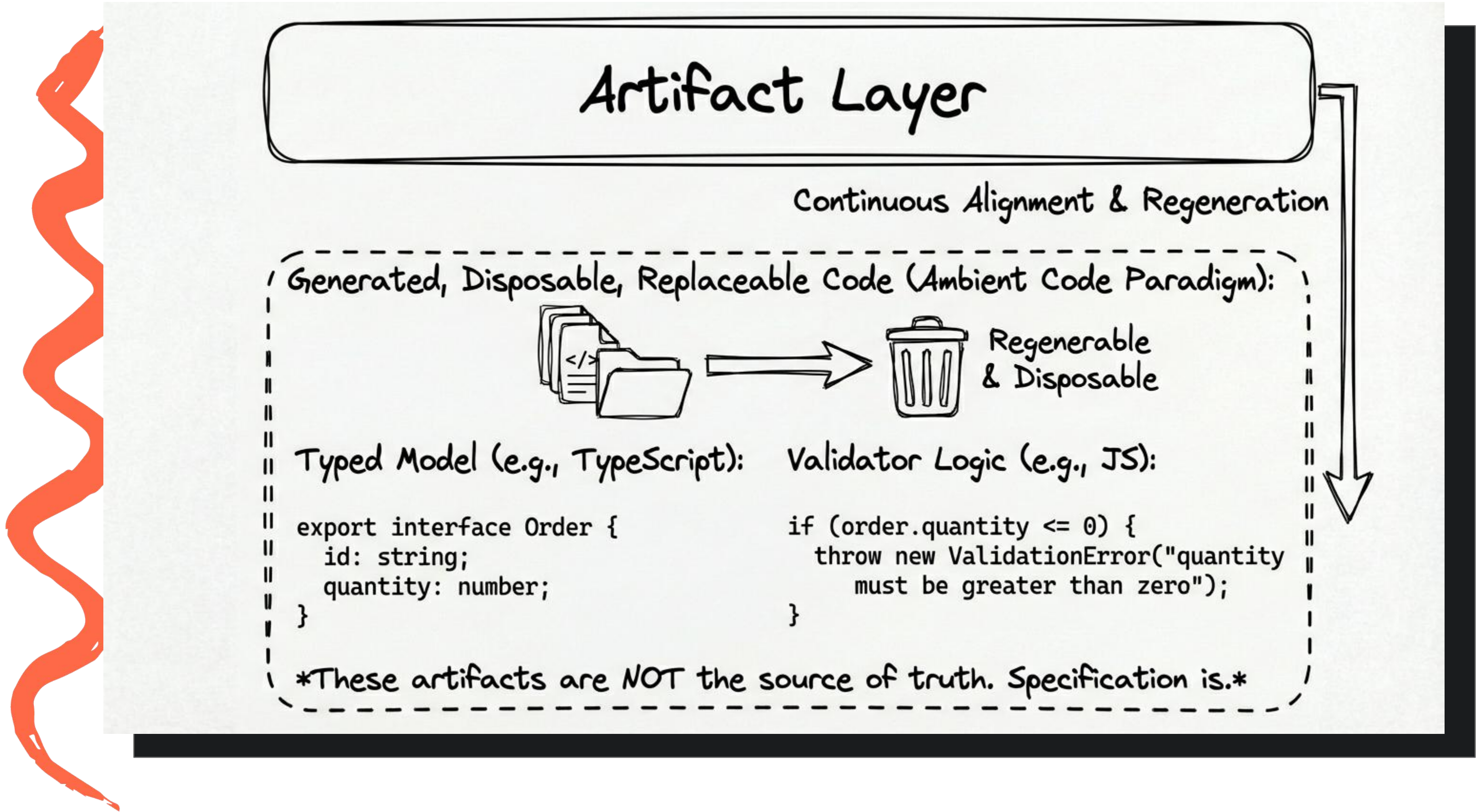
Specification layer



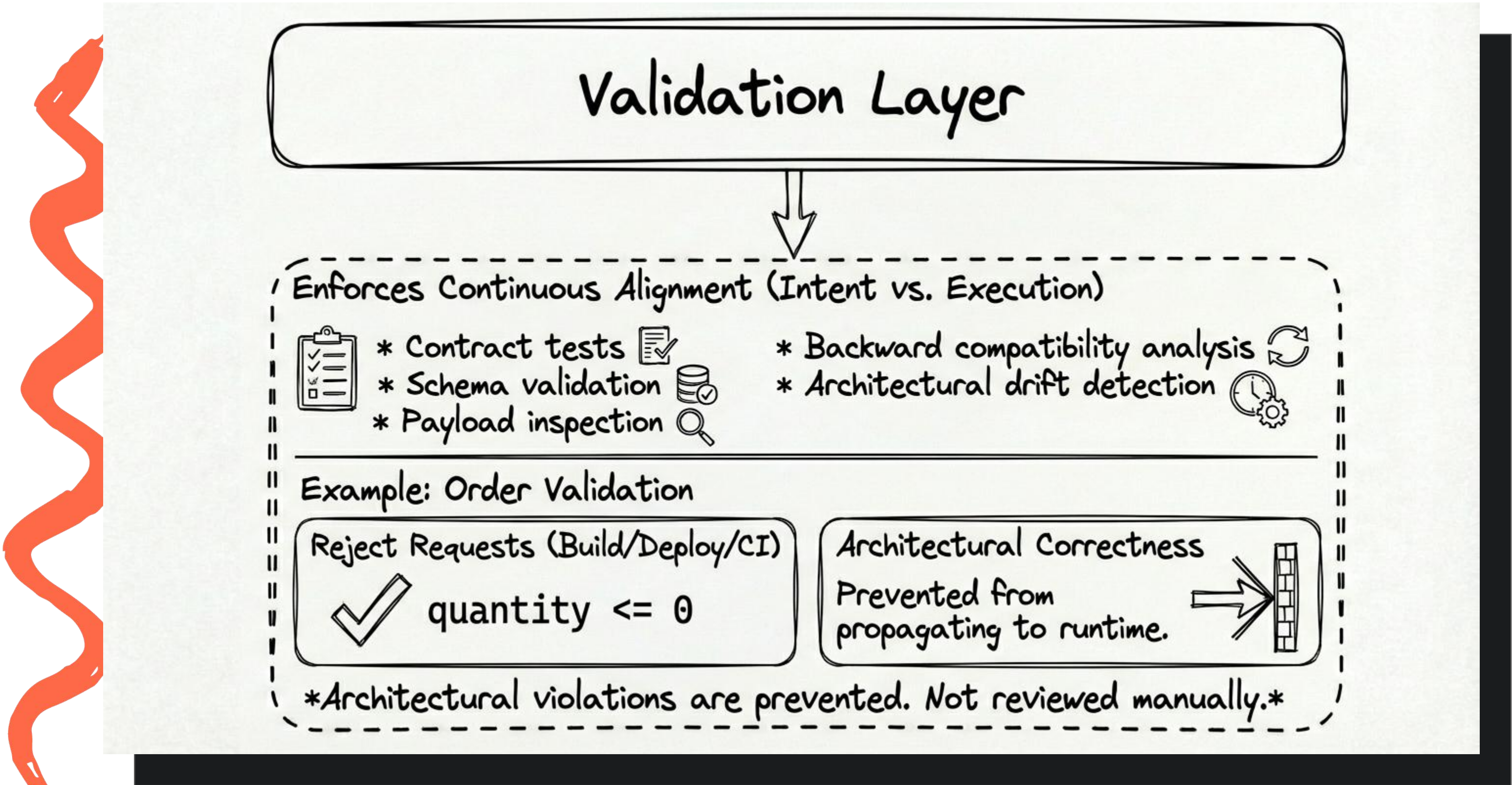
Generation layer



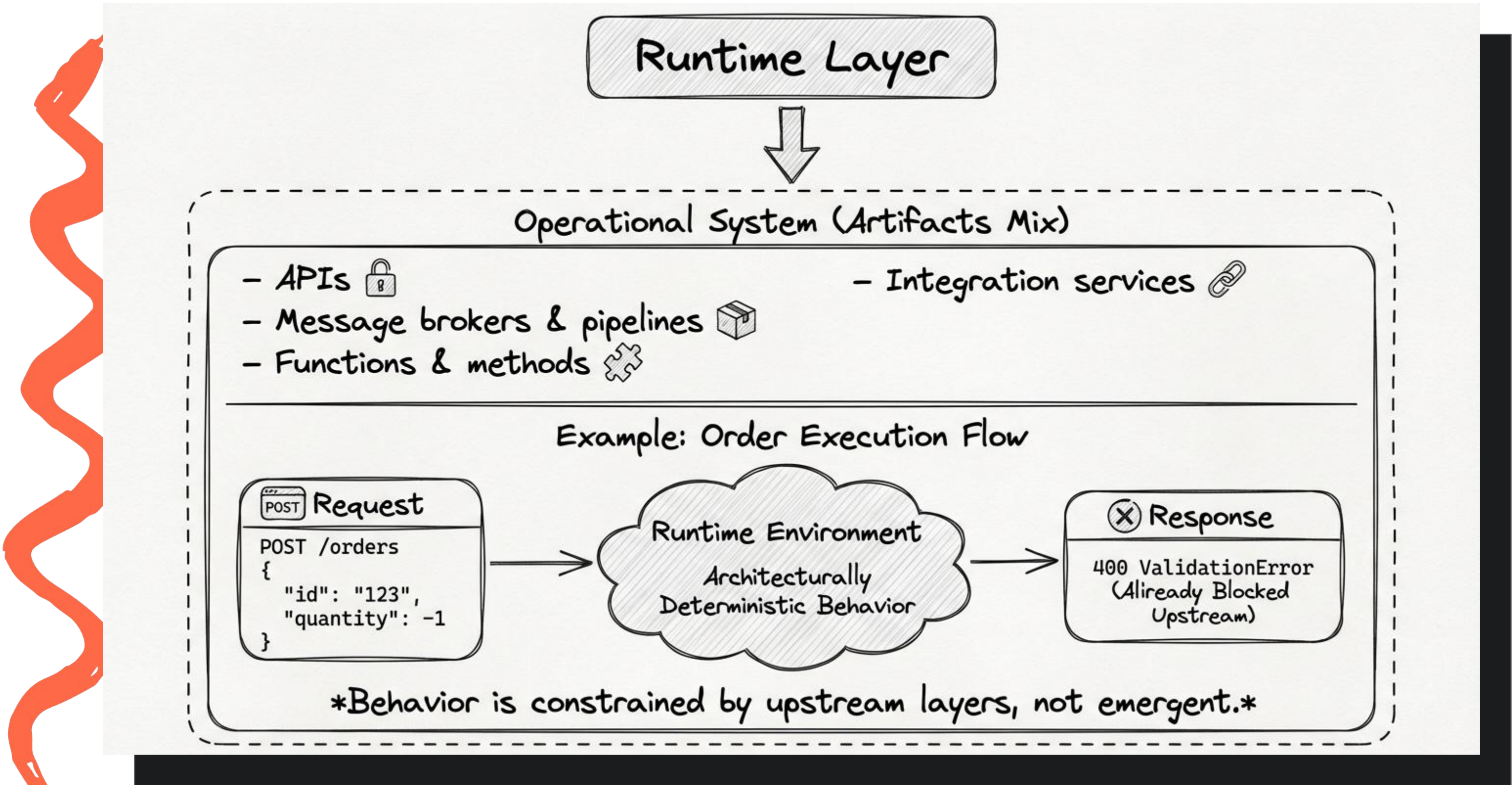
Artifact layer



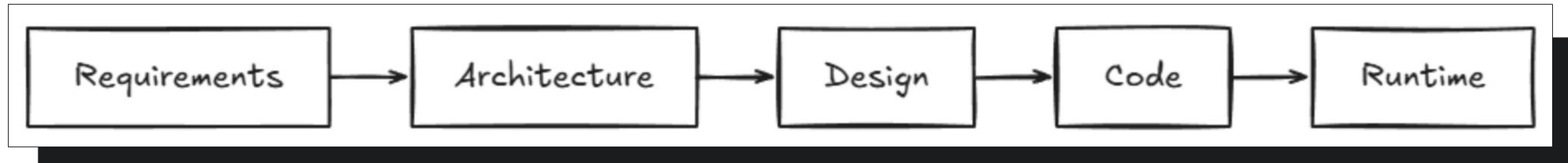
Validation layer



Runtime layer

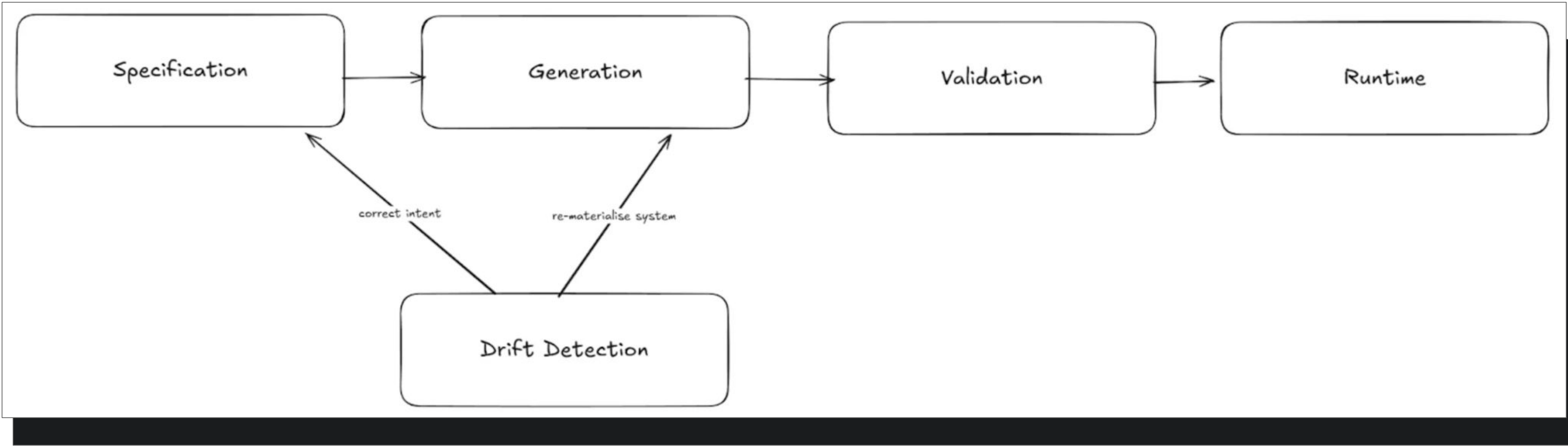


Architectural inversion



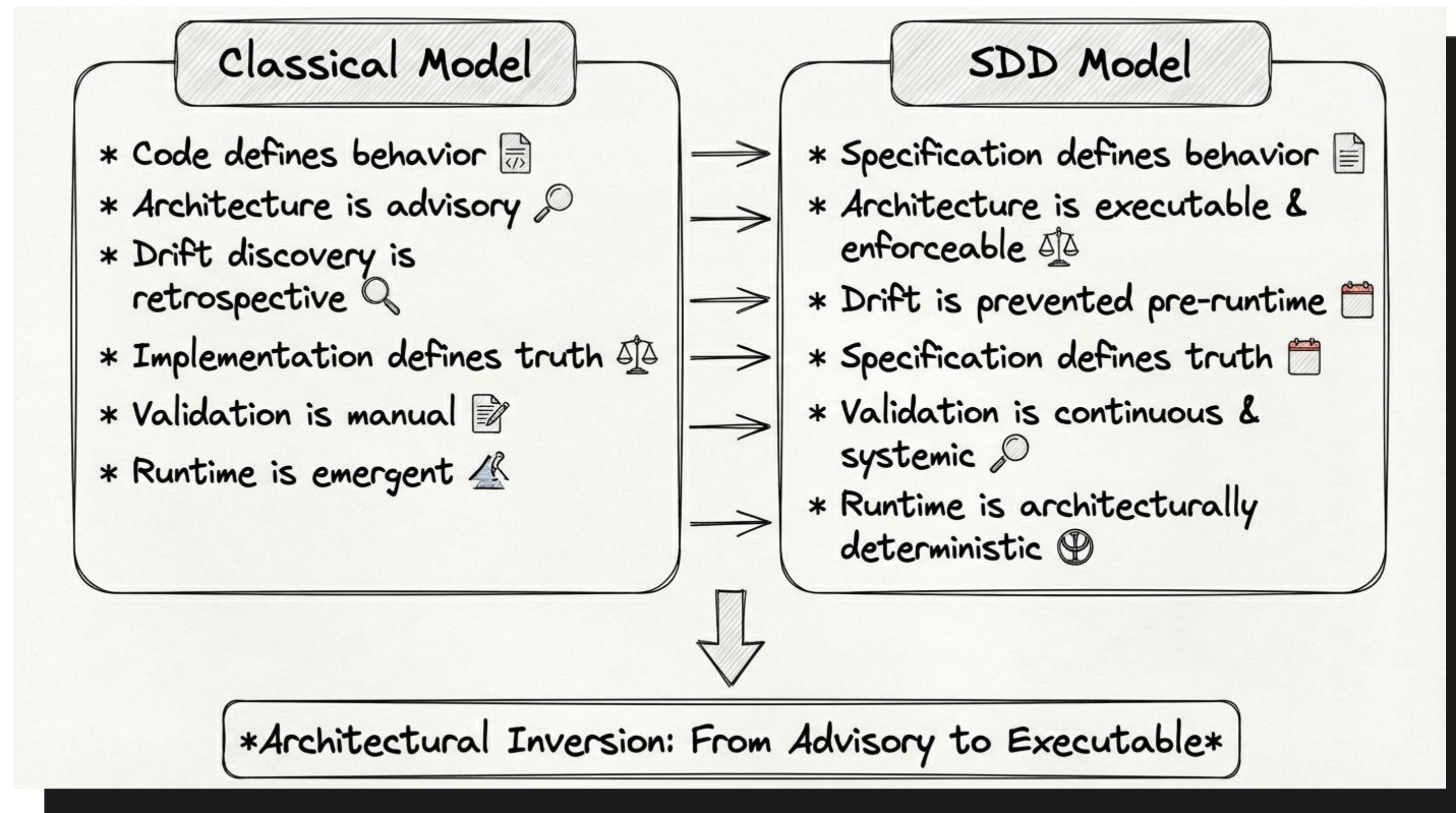
Традиционный взгляд на архитектуру

Architectural inversion

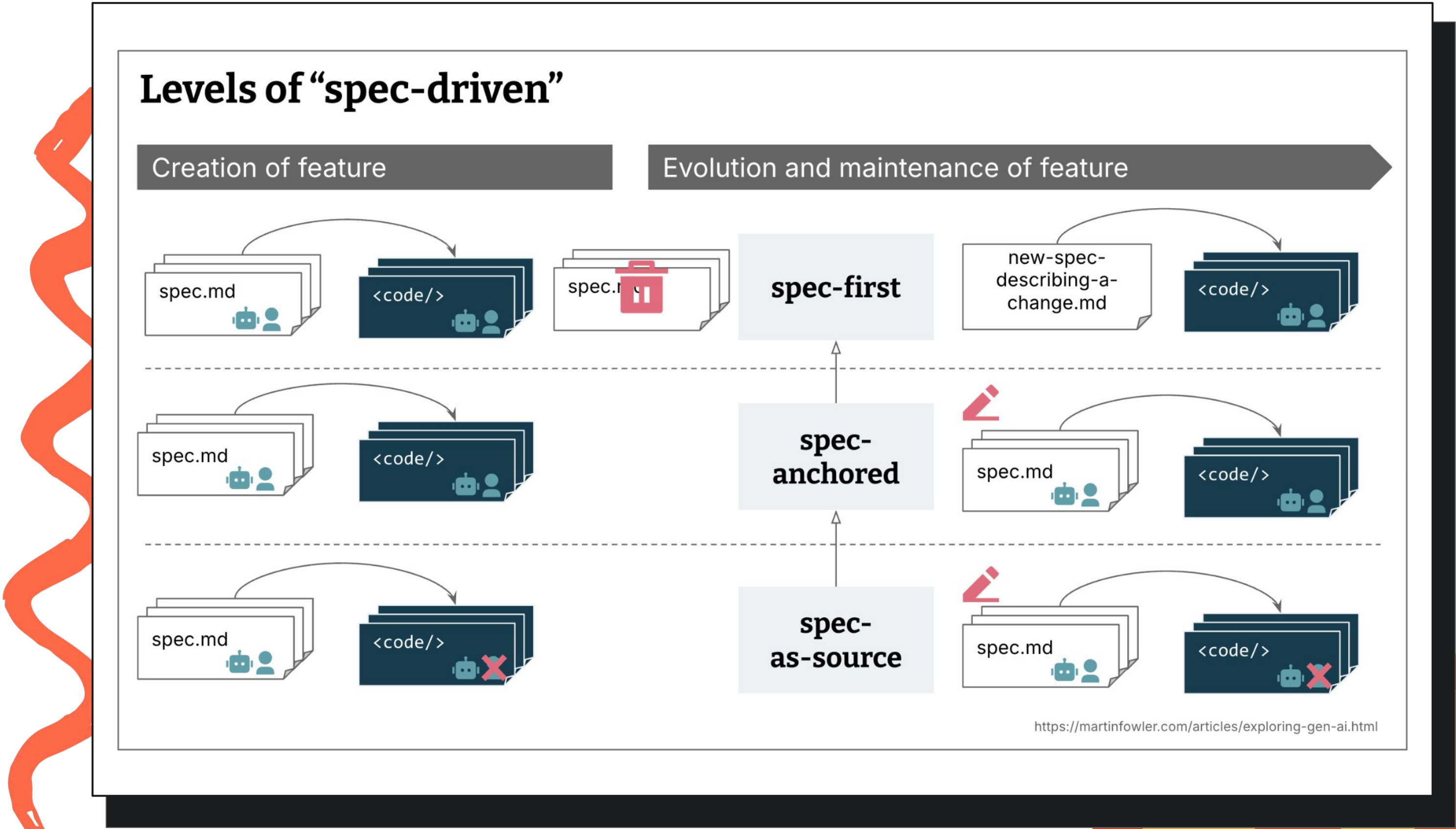


Нетрадиционный взгляд на архитектуру

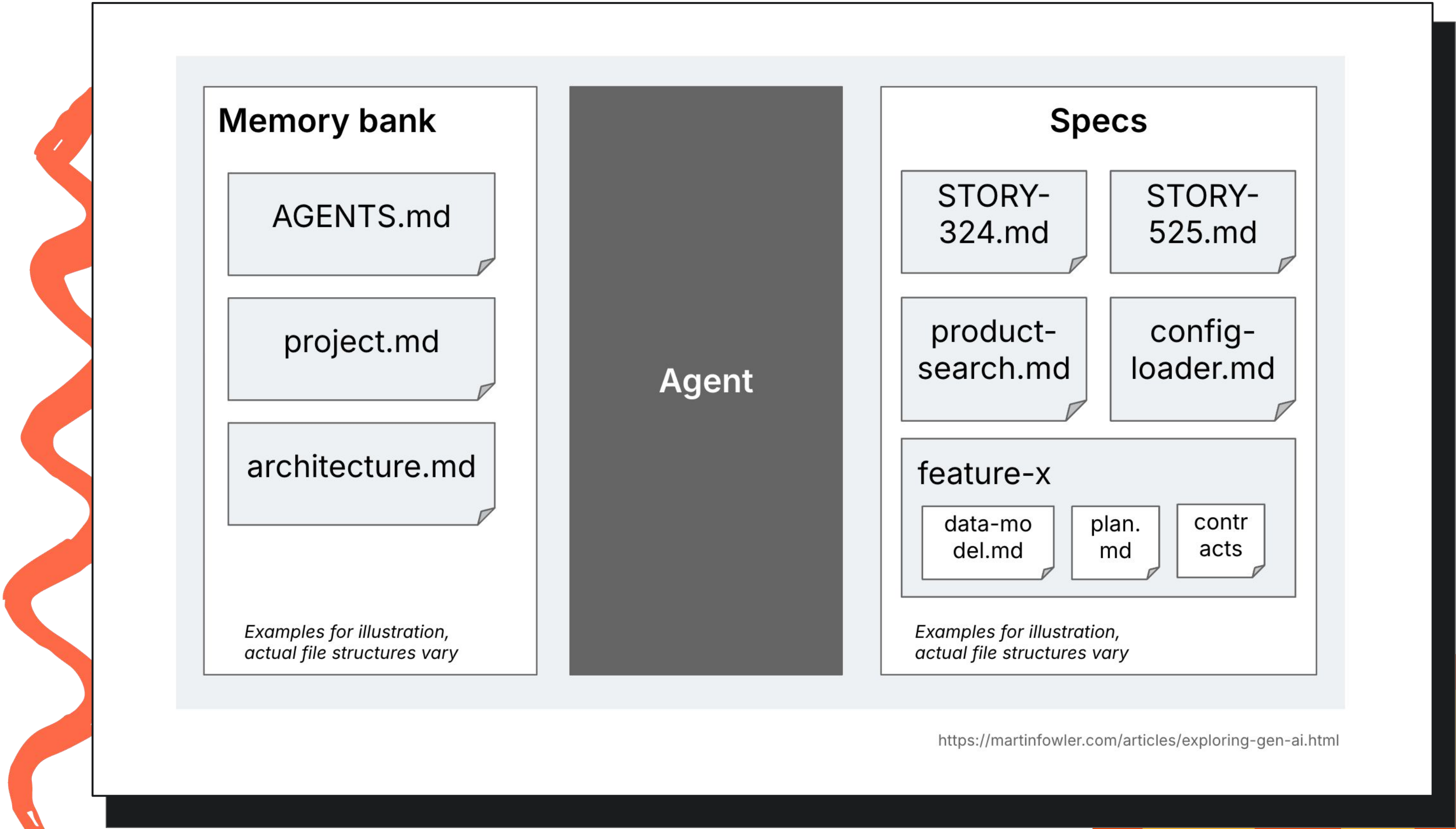
Summary



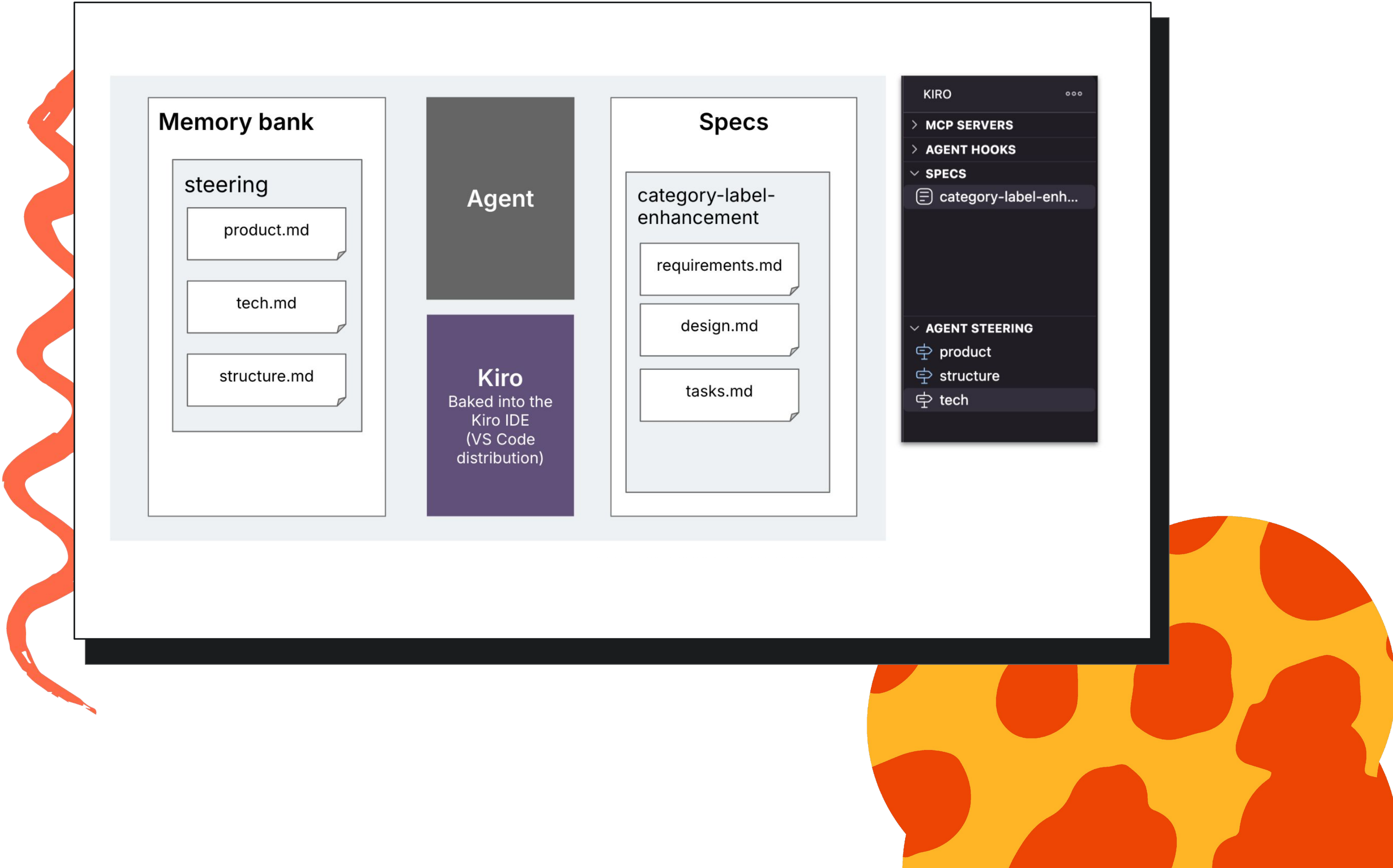
Детали реализации разных типов SDD



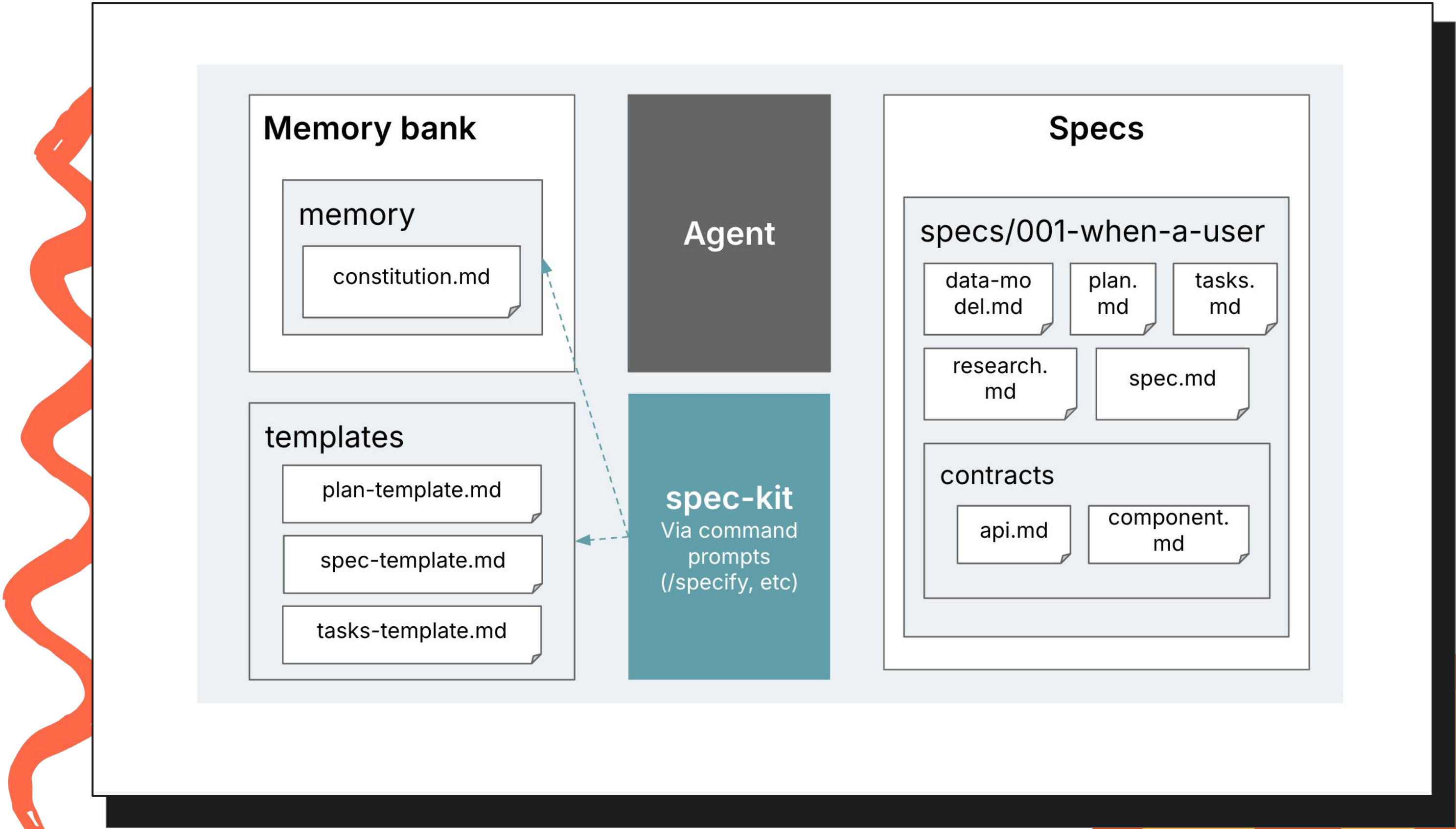
Общий вариант



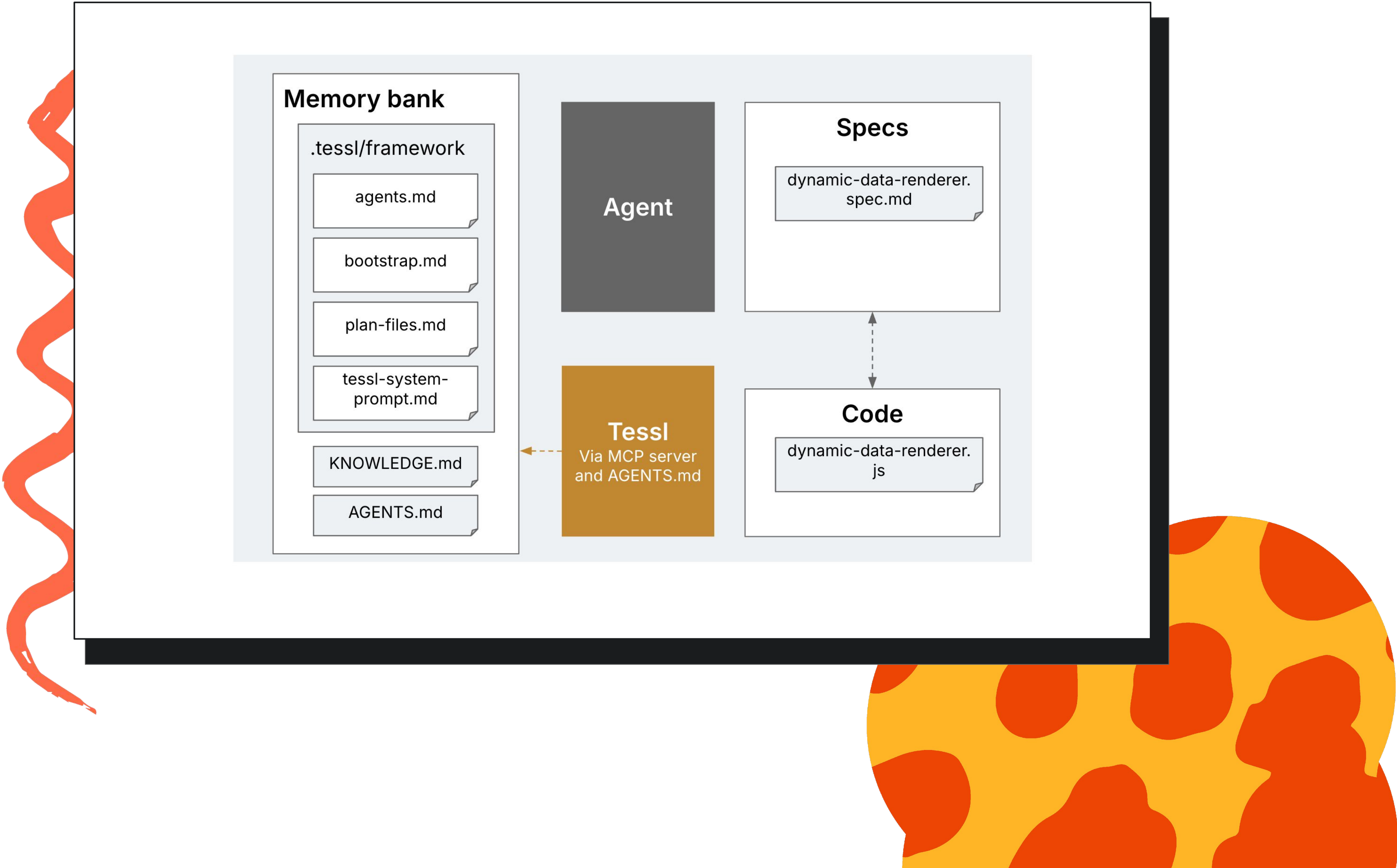
Kiro



Spec-kit



Tessl



РЗ. Не все так радужно

2



2

3

4

Overhead: спеки vs код



Colin Eberhardt (Scott Logic) протестировал
Спец Kit на реальной задаче:

SDD (Спец Kit)

33 МИН

2 577 строк markdown
→ 689 строк кода

Iterative prompting

8 МИН

~1 000 строк кода
без деградации качества

~4x

медленнее с SDD

67%

команд сталкиваются
с доп. отладкой при
внедрении SDD

Источники: Colin Eberhardt, «Putting Spec Kit Through Its Paces», Scott Logic Blog, Nov 2025
SoftwareSeni, «Spec-Driven Development in 2025», Sep 2025 (67% extra debugging)

Waterfall Strikes Back

Marmelab Blog, Nov 2025

SDD напоминает Waterfall — массивная документация перед кодом, чтобы разработчики просто «переводили» спеки.

Разработчики давно перестали быть просто исполнителями. Big Design Up Front доказано не работает, потому что накапливает гипотезы.

Thoughtworks / Liu Shangqi, Dec 2025

Генерация кода из спеки через LLM недетерминирована — это создаёт проблемы для обновлений и поддержки.

Опытные разработчики считают, что чрезмерно формализованные спеки создают лишние проблемы — как в ранние годы waterfall.



ThoughtWorks Technology Radar Vol.33: текущие SDD-воркфлоу слишком ригидны и не масштабируются.

Spec Ambiguity & Drift

“ Arcturus Labs

Естественный язык в спеках неточен и неоднозначен. Агент выполнит спеку буквально, но результат не совпадёт с ожиданиями.

“ B. Böckeler / martinfowler.com

Kiro превратил мелкий баг в 4 user stories с 16 acceptance criteria. Workflow как кувалда для гвоздя.

Спец ≠ Reality: почему спеки дрейфуют

«Вместо снижения overhead, SDD часто удваивает его. Каждое обновление — documentation debt под видом инженерной дисциплины» — [Isoform.ai](https://isoform.ai)

«Спец не отражают всего контекста. Edge cases появляются только при использовании, проблемы — под нагрузкой» — [DEV Community](https://devcommunity.ru)

Источники: arcturus-labs.com «Why SDD Breaks at Scale» • martinfowler.com «Understanding SDD: Kiro, spec-kit, Tessel» • isoform.ai «The Limits of SDD»

Тулинг и безопасность

Отсутствие единого подхода

Böckeler (Thoughtworks):

Все три инструмента называют себя SDD, но они совершенно разные. Ни один из них не подходит для большинства реальных задач.

TechChannel:

Спеки от AI-агентов часто раздуты — с ненужным многословием и допущениями. Приходится вручную урезать.

Нет стандарта формата. Kiro, Spec-kit, Tessler, OpenSpec — все разные.

AI-код и безопасность (ArXiv 2025)

9.8%–42.1%

уязвимого кода генерируют LLM
(ArXiv: Guiding AI to Fix Its Own Flaws, 2025)

32.8%

Python-кода от Copilot содержит уязвимости (Fu et al.)



SDD спеки не решают проблему безопасности сгенерированного кода

Когда использовать SDD?

When to Apply the SDD Framework?

SDD trades upfront spec effort for downstream clarity. Use it when cost of ambiguity > cost of documentation.

✓ WHEN TO USE SDD (High Value)

- Complex Features: Significant ambiguity.
- Heuristic: 5+ Story Points (Medium/High risk).
- Cross-Team Alignment needed.
- Critical Path: Expensive to fix later.

✗ WHEN TO AVOID SDD (High Overhead)

- Trivial Tasks: Typos, minor UI tweaks.
- Low complexity (<5 SP): "Just do it".
- Prototyping/Spikes: Exploration only.
- When specs take longer than code.

The "Blueprint" Metaphor



Building a Doghouse
(No detailed blueprint needed.
Just build it.)

<-- VS -->



Constructing a Skyscraper
(Detailed specs are essential
for safety and success.)

Открытые вопросы

1

**Спец или Code — что является source of truth?
Thoughtworks: «вопрос ещё нужно исследовать»**

2

**Как держать spec-code-tests треугольник
в синхронизации? (D. Breunig, Mar 2026)**

3

**Для каких задач SDD оправдан, а для каких —
overkill? Kiro → баг = 4 user stories (Böckeler)**

Summary

1



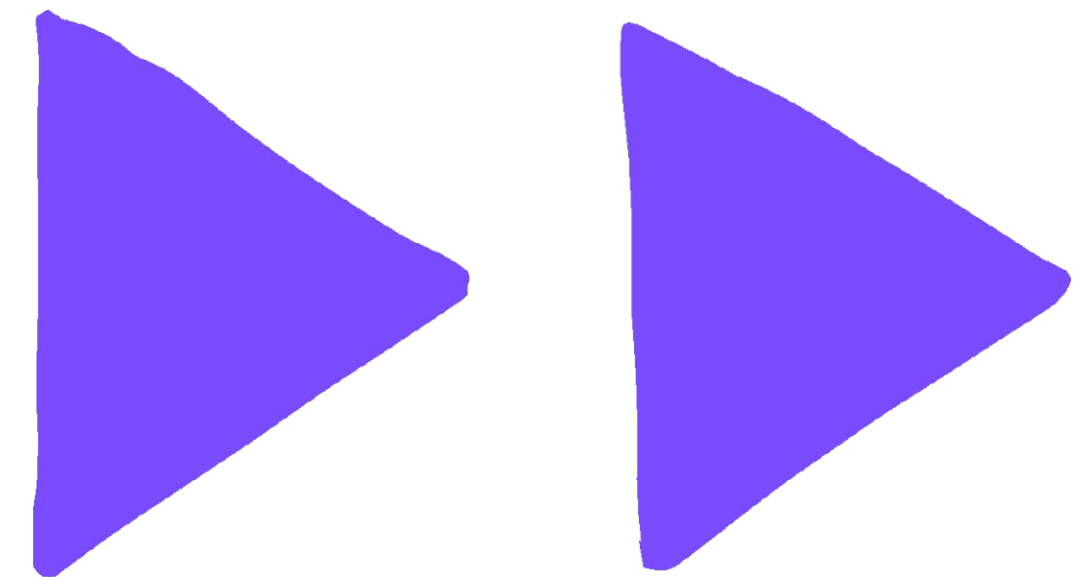
2

3

4

В будущем:

- Фундаментальные знания остаются основой разработки
- Новый виток arch-first/сpec-first
- Новое направление валидации - spec-review



Статья про контекст менеджмент и когнитивное программирование

